Aerial Grounding on Jetson Orin Nano

A deployable vision-language model — how it was built and why

v2 "Principled Rebuild" — Part II of the Jetson edge-LLM thesis

By Javier F. Dibo Gómez

Table of contents

1. Executive summary

2. What the model does

3. The starting point and the two binding constraints

3.1 Constraint 1 — the deployment-fidelity gap

3.2 Constraint 2 — the tiny-object resolution ceiling

4. Methodology: a gated, fidelity-first workflow

4.1 The shared contract — the spine of the whole system

5. Phase 0 — backend fidelity and model selection

6. Phase 1 — dataset audit

7. Phase 2 — resolution strategy

8. Phase 3 — training

9. Phase 4 — export and deployment

9.1 Export

9.2 The fidelity disambiguation — the headline result

9.3 The 8 GB memory fix

10. Serving and demo architecture

11. Key decisions and their rationale

12. Results, limitations, and honesty notes

13. How to reproduce

14. Conclusion

List of tables

Table 1.1: The deployed model at a glance.

Table 2.1: A real prediction from the deployed Q8_0 model versus ground truth.

Table 3.1: The Part-I fidelity collapse, discovered post-hoc.

Table 4.1: The five gated phases and their gates.

Table 4.2: The single source of truth every component imports.

Table 5.1: Base-model selection probe (RefCOCO val, n = 100, identical contract).

Table 6.1: RefDrone audit findings that shaped the training set.

Table 7.1: Resolution ladder on the frozen base model (RefDrone val, n = 439).

Table 8.1: Phase-3 training configuration.

Table 8.2: Phase-3 result versus the frozen-base baseline (same n = 439).

Table 9.1: Exported artifacts.

Table 9.2: Deployed fidelity — the central Phase-4 measurement.

Table 9.3: The whole point, side by side.

Table 10.1: The three ways to demo the deployed model.

Table 11.1: The decisions that shaped the model.

Table 12.1: Final scorecard.

List of code blocks

Code block 9.1: The server flags that keep memory flat on an 8 GB device.

Code block 10.1: What happens on one demo request, end to end.

Code block 13.1: Reproducing the deployed model from the trained checkpoint.

1. Executive summary

This document describes a working **aerial visual-grounding model** and the deliberate, gated process that produced it. Given a top-down drone image and a natural-language phrase ("the white car near the building"), the model returns a single bounding box locating that object. It runs **on the edge device itself** — an NVIDIA Jetson Orin Nano with only 8 GB of unified memory — not on a server.

The headline outcome is twofold:

- **Accuracy:** full-validation **IoU@0.25 = 62.6%** (n = 439) for the deployed 8-bit model — roughly **3.1× the previous exploratory result** (19.5%) on the same task.
- **Deployment fidelity:** the model lost **no measurable accuracy** moving from the training GPU to the edge device (runtime gap −2.7 pp, quantization gap −0.5 pp). The earlier exploratory arc had lost ~30 points on the same step. Eliminating that loss is the central methodological contribution.

The work is the second part ("v2 — Principled Rebuild") of a master's thesis on running local language models on edge hardware. Part I was exploratory and discovered the constraints; Part II is the deliberate rebuild organised around them.

Property	Value
Task	Referring-expression grounding (one box per phrase)
Domain	Aerial / drone imagery (RefDrone over VisDrone)
Base model	Qwen2-VL-2B-Instruct (2 B parameters, vision+language)
Adaptation	LoRA fine-tune (18.5 M params, 0.83% of the model)
Input resolution	1024 px long edge
Deployment format	GGUF, Q8_0 quantization (1.65 GB)
Inference engine	llama.cpp with CUDA, on the device
Device	Jetson Orin Nano 8 GB, 15 W, clocks locked
Accuracy (deployed)	IoU@0.25 = 62.6%, parse rate 100% (n = 439)

Table 1.1: The deployed model at a glance.

2. What the model does

The model is a **referring-expression grounder**. It takes two inputs and produces one output.

- **Input 1 — an image:** a top-down aerial photograph (drone footage).
- **Input 2 — a phrase:** a free-text description of one object or group ("the blue cars on the left side of the street").
- **Output — a bounding box:** four integers `[x1, y1, x2, y2]` in a normalized 0–1000 coordinate space, emitted as a small JSON object.

It is *not* a general object detector. It does not enumerate every object in the scene; it localizes the **one** thing the phrase refers to. This is a harder, more language-driven task than fixed-class detection, and it is the capability a natural-language drone interface needs ("point the camera at the truck by the gate").

Phrase	Predicted box	Ground-truth box	Overlap
"The black cars at the intersection."	[266, 476, 346, 644]	[268, 481, 343, 652]	IoU ≈ 0.9
"The blue cars on the left side of the street."	[324, 267, 341, 299]	[325, 263, 342, 294]	IoU ≈ 0.9

Table 2.1: A real prediction from the deployed Q8_0 model versus ground truth.

The accuracy metric used throughout is **IoU@0.25**: the fraction of predictions whose box overlaps the ground truth with an Intersection-over-Union of at least 0.25. This is a deliberately *coarse* localization threshold — it credits "the object is roughly there", which is the right

bar for a steerable-camera interface, and an honest one to report rather than cherry-picking a stricter number.

3. The starting point and the two binding constraints

Part I of the thesis reached a real result (aerial IoU@0.25 = 19.5%) but the *process* was exploratory, and it surfaced two findings that turned out to be the real reasons progress stalled. The v2 rebuild is organised entirely around them.

3.1 Constraint 1 — the deployment-fidelity gap

In Part I, an exported skill was measured at each step of the journey to the device and lost accuracy catastrophically — and the loss was only discovered *after* training was finished.

Stage	Accuracy	Loss
Original model (PyTorch, bf16)	85%	—
Converted to GGUF F16	62%	−23 pp (runtime / image-preprocessing)
Quantized to Q8_0	55%	−7 pp (quantization)

Table 3.1: The Part-I fidelity collapse, discovered post-hoc.

The dominant loss (−23 pp) was **not** quantization — it was a divergence between how the original framework and the edge inference engine preprocessed images. The lesson: **measure backend fidelity before spending GPU time on training**, and pick the model by how well it survives deployment, not by its benchmark score in the lab.

3.2 Constraint 2 — the tiny-object resolution ceiling

Aerial objects are small: 5–30 px in the source image. After the standard 512 px resize through a **frozen** vision encoder, they shrink to 2–11 px — too few "visual patches" for the model to describe. No amount of language-model fine-tuning recovers detail the encoder already discarded. The lesson: **treat input resolution as an explicit, pre-registered experimental variable**, not a default.

4. Methodology: a gated, fidelity-first workflow

v2 is designed **backwards from deployment** and **de-risks cheaply before spending GPU**. It proceeds in five gated phases; each phase has a pass/fail gate and may not begin until the prior gate is green *and* documented in the lab notebook in the same step.

Phase	Purpose	Gate
0 — Backend fidelity	Choose the model by deployment fidelity	Spine chosen by the numbers; gap quantified
1 — Dataset audit	Understand the data before training	Target well-posed; object-size distribution known
2 — Resolution	Pick input resolution deliberately	One resolution chosen and justified
3 — Train	One config-driven LoRA loop	IoU@0.25 ≥ 20%, parse ≥ 90%, non-degenerate
4 — Export & deploy	GGUF on the device, fidelity gate	Deployed IoU within the fidelity budget

Table 4.1: The five gated phases and their gates.

4.1 The shared contract — the spine of the whole system

Part I failed partly because five different scripts each defined their own copy of the prompt, the output parser, and the metric — and they drifted apart. v2 fixes this with **one module**, `grounding/contract.py`, that defines the canonical prompt, parser, and metrics, and is imported *everywhere*: the fidelity probe, the trainer, the exporter, the on-device backend, and the demo.

Symbol	Role
`GROUNDING_PROMPT`	The exact instruction text wrapped around every query
`parse_bbox`	Turns the model's text output into `[x1,y1,x2,y2]` or None
`iou`	Intersection-over-Union between two boxes
`center_std`	Spread of predicted box centres — a mode-collapse detector
`COORD_SCALE = 1000`	The normalized coordinate range

Table 4.2: The single source of truth every component imports.

This guarantees the prompt the model was *trained* on is byte-identical to the one the GUI *sends* months later. A pytest suite locks the prompt text byte-for-byte so it can never drift again.

5. Phase 0 — backend fidelity and model selection

The goal of Phase 0 was to choose the base model **by deployment fidelity**, before any fine-tuning. We built a backend-agnostic evaluation harness so the same dataset, prompt, parser, and metric could be run against three backends — the training framework (HF), the local edge engine (GGUF), and the device itself (Jetson) — behind one interface.

Two filters decided the model:

- **Deployment-first elimination:** the pinned edge engine had zero projector support for some candidates (PaliGemma 2, Florence-2). They were disqualified *before download* — there is no point training a model you cannot serve.
- **Measured fidelity, base-vs-base:** the survivors (SmolVLM-500M and Qwen2-VL-2B) were probed on identical data through the contract path.

Candidate	Grounding-native?	HF→F16 fidelity gap	Verdict
SmolVLM-500M (Idefics3)	output collapsed	−16 pp (measured in self-check)	rejected
Qwen2-VL-2B	healthy boxes	**−2 pp**	**chosen**

Table 5.1: Base-model selection probe (RefCOCO val, n = 100, identical contract).

Qwen2-VL-2B was chosen on three axes: it is grounding-capable, it has native dynamic resolution (which directly attacks Constraint 2), and its edge-engine preprocessing matches its training preprocessing (~8× better fidelity than the incumbent — directly attacking Constraint 1).

6. Phase 1 — dataset audit

Before any GPU run, the dataset (RefDrone, built over the VisDrone aerial benchmark) was audited against the canonical schema. The audit immediately found that the raw target was **ill-posed**: captions referred to *multiple* boxes on average, which is unlearnable as a single-box task.

Finding	Value	Consequence
Mean boxes per caption (raw)	3.80	Target is ill-posed as single-box
Well-posed fraction (one box)	33.2%	Filter to a clean subset
Resulting training set	4101 train / 439 val	The set actually used
Median aerial object size @512 px	~16 px	Quantifies Constraint 2
Bottom-quartile object size @512 px	6–10 px	Near-unrecoverable at 512

Table 6.1: RefDrone audit findings that shaped the training set.

A `assert_well_posed` gate now refuses any future dataset whose single-box fraction is below a threshold — exactly the check that would have caught a Part-I failure mode for free.

7. Phase 2 — resolution strategy

With the model frozen (no training), the input resolution was swept over four values to isolate its effect. Resolution turned out to be the single most powerful lever in the entire project.

Max side (px)	IoU@0.25	Parse rate	Notes
512	4.1%	100%	objects too small
768	10.7%	100%	rising
1024	30.3%	91.8%	the elbow — chosen
1280	38.7%	92%	ceiling; held as a reserve lever

Table 7.1: Resolution ladder on the frozen base model (RefDrone val, n = 439).

A **9.4× swing in accuracy from resolution alone, with frozen weights**, reframes the Part-I 19.5% result as resolution-starved rather than under-trained. **1024 px** was chosen as the elbow: it captures ~78% of the 1280 ceiling, clears the project's 20% gate zero-shot, and fits the Jetson's 8 GB memory budget. 1280 px was kept in reserve as an accuracy lever in case training fell short — it was never needed.

8. Phase 3 — training

A single, config-driven LoRA training loop (not the per-stage script forks of Part I) fine-tuned the chosen model on the audited data at the chosen resolution.

Hyper-parameter	Value
Base model	Qwen2-VL-2B-Instruct
Method	LoRA, rank 16 / alpha 32, on the language attention + MLP layers
Vision encoder	frozen
Trainable parameters	18.5 M (0.83% of the model)
Resolution	1024 px long edge
Learning rate	2e-4
Epochs	3
Effective batch size	16
Hardware	RTX 3090, 24 GB

Table 8.1: Phase-3 training configuration.

The result cleared the gate at the first epoch and finished well above it.

Configuration	IoU@0.25	Parse rate	Centre spread
Frozen base @ 1024 (Phase 2)	30.3%	91.8%	healthy
Fine-tuned @ 1024 (Phase 3)	**59.5%**	**100%**	healthy (215)

Table 8.2: Phase-3 result versus the frozen-base baseline (same n = 439).

The gain decomposes cleanly into two independent factors that were *each measured*: resolution (512→1024 on the base: 4.1%→30.3%) and fine-tuning (30.3%→59.5%). Neither reserved lever (1280 px, data augmentation, warm-start) was needed.

9. Phase 4 — export and deployment

This phase closes the loop the whole design was built backwards from: take the trained model to the actual device and prove it keeps its accuracy.

9.1 Export

The merged checkpoint was converted to GGUF (the format llama.cpp reads) at two precisions, plus the shared vision projector.

Artifact	Precision	Size
Model	F16	3.09 GB
Model	Q8_0 (8-bit)	1.65 GB
Vision projector (mmproj)	F16	1.33 GB

Table 9.1: Exported artifacts.

Because the vision encoder was frozen during training, the exported projector is **bit-equivalent to the base model's** — one projector serves both base and fine-tune, nothing extra to ship.

9.2 The fidelity disambiguation — the headline result

Every backend was scored over the **full** validation set (n = 439) through the *same* contract path, on the device, with the local converter and the device using the *same pinned* engine commit (so engine version is not a confound — only the hardware and the quantization differ).

Backend	IoU@0.25	Parse rate	Gap
HF bf16 (reference)	59.5%	100%	—
GGUF F16 (Jetson, CUDA)	62.2%	100%	−2.7 pp runtime (F16 *beats* reference)
GGUF Q8_0 (Jetson, CUDA)	62.6%	100%	−0.5 pp quant (vs F16)

Table 9.2: Deployed fidelity — the central Phase-4 measurement.

The Part-I catastrophe **does not reproduce**. Both deployed quants land *above* the reference, within sampling noise on n = 439. The honest reading is "no measurable runtime loss", not "GGUF improves the model" — and that absence of a gap is precisely the thesis claim.

	Runtime gap (→F16)	Quant gap (→Q8_0)	Net
Part I (SmolVLM/ Idefics3)	−23 pp	−7 pp	catastrophic, post-hoc
v2 (Qwen2-VL-2B)	**−2.7 pp**	**−0.5 pp**	**no loss, pre-characterised**

Table 9.3: The whole point, side by side.

Q8_0 was accepted as the deployment artifact: half the size of F16, indistinguishable accuracy, and it fits the 8 GB unified memory with headroom.

9.3 The 8 GB memory fix

The first device run crashed mid-evaluation: the engine's default 8 GB prompt cache plus automatic multi-slot parallelism exhausted the

unified memory and the OS killed the server. The fix was to run a single slot with the prompt cache disabled.

```
llama-server -m model.gguf --mmproj mmproj.gguf \
  -ngl 99 -c 4096 \
  -np 1 --cache-ram 0 --no-cache-idle-slots \
  --port 18080 --host 127.0.0.1
```

Code block 9.1: The server flags that keep memory flat on an 8 GB device.

With these flags the server holds a stable ~5.8 GB resident set with ~1.3 GB of headroom across all 439 samples.

10. Serving and demo architecture

The deployed model is fronted by a small serving stack and two demo surfaces, all sharing the same contract path so a live demo and a benchmark number can never diverge.

```
Browser (HTML + vanilla JS)
   |  image as base64 JSON  ──POST /infer──►
Python http.server  (grounding/deploy/gui.py, on the PC)
   |  GROUNDING_PROMPT + image  ──►  via SSH tunnel
llama-server  (on the Jetson, CUDA, Q8_0)
   ◄── '{"bbox": [324, 267, 341, 299]}'
parse_bbox → box → PIL draws it → back to the browser
```

Code block 10.1: What happens on one demo request, end to end.

The serving plumbing (`grounding/deploy/serve.py`, `grounding/eval/backends.py`) boots `llama-server` on the device over `ssh jetson`, opens an `ssh -N -L` port-forward tunnel, and talks to it as if it were local. It tears the tunnel down and kills the remote process on exit.

Two design choices keep the demo dependency-free and reproducible:

- **No web framework.** The GUI uses only the Python standard library plus Pillow (already present); no Flask or Gradio, so the lock-pinned environment stays untouched.
- **Warm backend.** The model is booted once at startup and reused for every request, so only the first query pays the load cost.

Surface	Command	Best for
Browser GUI	`python -m grounding.deploy.gui`	Live, interactive demo
One-shot CLI	`python -m grounding.deploy.demo --image … --caption …`	A single annotated image
Full evaluation	`python -m grounding.eval.run --backend jetson …`	Reproducing the headline number

Table 10.1: The three ways to demo the deployed model.

11. Key decisions and their rationale

Every non-trivial choice was logged with its reasoning in the project decision log. The most consequential are summarised here.

Decision	Why	Trade-off accepted
Choose the model by deployment fidelity, before training	Part I lost 23 pp post-hoc; fidelity, not benchmark score, is what survives	Extra up-front probe work before any training
Qwen2-VL-2B over SmolVLM-500M	Grounding-native, native dynamic resolution, ~8× better fidelity	Larger model (2 B vs 0.5 B) to fit on 8 GB
Filter dataset to well-posed single-box subset	Multi-box target is unlearnable as single-box	Training set shrinks to 4101
Resolution 1024 px (the elbow)	78% of the ceiling, clears the gate, fits 8 GB	Leaves some accuracy on the table vs 1280
LoRA with frozen vision encoder	Cheap, stable, mmproj reuse; resolution carries the visual gain	Tiny objects remain encoder-limited
Q8_0 as the deployment artifact	Half the size, indistinguishable accuracy	None measurable
Single-slot, no prompt cache on device	The default OOM-kills the server on 8 GB	No multi-request concurrency
Dependency-free stdlib GUI	Keep the lock-pinned environment reproducible	Hand-written HTML instead of a framework

Table 11.1: The decisions that shaped the model.

12. Results, limitations, and honesty notes

Metric	Value
Deployed accuracy (Q8_0, IoU@0.25)	62.6% (n = 439)
Parse rate	100%
Improvement over Part I	~3.1× (19.5% → 62.6%)
Runtime fidelity loss	−2.7 pp (none, within noise)
Quantization fidelity loss	−0.5 pp (none, within noise)
Deployed model size	1.65 GB
Device	Jetson Orin Nano 8 GB, 15 W

Table 12.1: Final scorecard.

The result is reported honestly, with its limits stated:

- **IoU@0.25 is a coarse threshold.** It credits rough localization; ~1 in 3 queries still misses even at this bar, and tighter localization (IoU@0.5) would score lower.
- **The resolution ceiling still binds.** The smallest objects (6–10 px) are near-unrecoverable through the frozen encoder; pushing further would mean 1280 px input or unfreezing the encoder.
- **It is a single-box referring grounder**, not a multi-object detector.
- **Evaluated on the well-posed validation subset**, not arbitrary live drone footage.
- **"Deployed beats reference" is noise, not a gain** — the claim is the *absence* of the Part-I gap, not that quantization improves the model.

13. How to reproduce

The full pipeline is driven through a Makefile and a pinned, locked environment. The core steps:

```
source .venv-ft/bin/activate
# 1) export the trained checkpoint to GGUF F16 + Q8_0 +
mmproj
python -m grounding.export.to_gguf runs/v2/phase3-
refdrone-1024
# 2) push to the device and score the full validation set per
quant
python -m grounding.eval.run --backend jetson --dataset refdr
one --split val --n 0 \
  --model  /home/jfdg/grounding/phase3-refdrone-1024-
q8_0.gguf \
  --mmproj /home/jfdg/grounding/mmproj-phase3-refdrone-1024-
f16.gguf --max-side 1024
# 3) demo it in the browser
python -m grounding.deploy.gui   # open http://127.0.0.1:8000
```

Code block 13.1: Reproducing the deployed model from the trained checkpoint.

Provenance is plaintext and committed: every run writes a manifest capturing the git commit, the pinned engine commit, the locked dependency hash, and the exact configuration, so any number in this document can be traced to the run that produced it.

14. Conclusion

v2 set out to prove a single methodological claim: that the deployment-fidelity collapse which stalled Part I was avoidable by **choosing the model for the edge before training it**, and by **treating resolution as a first-class variable**. Both held. The resulting model grounds aerial referring expressions at IoU@0.25 = 62.6% — about three times the prior result — and it does so *on the 8 GB edge device itself, with no fidelity lost in deployment*. The engineering scaffolding (one shared contract, gated phases, per-run manifests, a locked environment) is what makes that claim reproducible rather than anecdotal, which is the real deliverable for the thesis.